

LangGraph.

Agents as state machines.

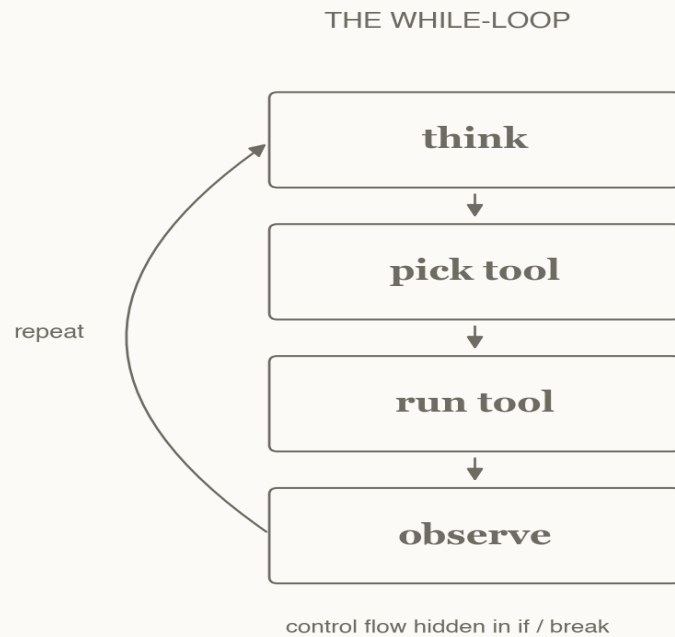
Where we are.

- Class 6 — agent from scratch: while-loop + tool dispatch.
- Everything since — RAG, fine-tuning, RL, memory.
- Today — the loop, formalized.

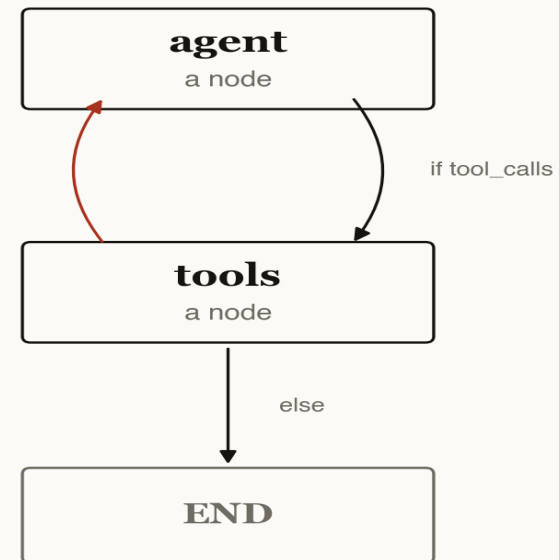
The callback — your Class 6 agent.

- while not done:
- think → pick tool → run tool → observe
- State lived in local variables.
- Control flow lived in if / break.

The reframe.



THE GRAPH



nodes = the work · edges = what's next · state = the whiteboard

Three primitives.

State

shared snapshot

Nodes

functions that read state, return updates

Edges

decide the next node

The Primitives

State, nodes, edges.

Six lines is the whole API surface.

State.

- A TypedDict schema.
-
- Nodes return partial updates.
-
- The graph merges them.

Reducers.

default

overwrite

Annotated[list, add]

append

add_messages

append + update by ID

MessagesState.

- Prebuilt state.
- One messages key.
- `add_messages` reducer baked in.
- Subclass to add fields.

Nodes.

- `def node(state) -> dict:`
- Optional: config, runtime.
- Return is a state update — not the full state.

Edges.

`add_edge(A, B)`

normal

`add_conditional_edges(A, router)`

conditional

`START / END`

the entry and exit

`multiple out-edges`

run in parallel

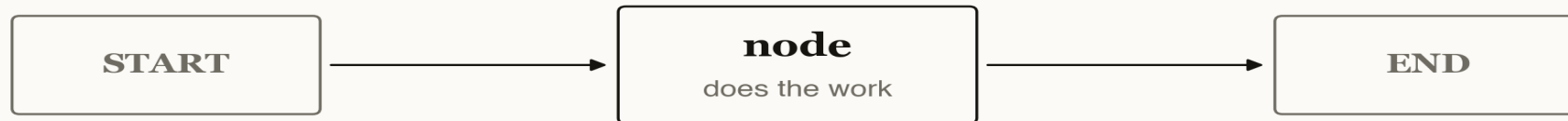
Compile.

- `builder.compile()`
- Runs structure checks.
- Attaches the checkpointer / store.
- Must compile before use.

The hello-world graph.

The hello-world graph

Six lines. This is the whole API surface.



```
g = StateGraph(MessagesState)
g.add_node("node", fn)
g.add_edge(START, "node")
g.add_edge("node", END)
app = g.compile()
```

Six lines. This is the whole API surface.

Runtime

Command, Send, super-steps.

Where persistence and control live.

Command.

`Command(update={...}, goto="next")`

update and route in one return

replaces

the router + the edge

Persistence = the checkpointer.

snapshot

at every super-step

keyed by

thread_id

backends

InMemorySaver → SqliteSaver → PostgresSaver

What persistence unlocks.

- Conversational memory.
 - Human-in-the-loop.
 - Time travel.
 - Fault tolerance.
-

Threads vs. store.

checkpointer

state within a thread

store

memory across threads

store, also

namespaced, optional semantic search

Interrupts.

- `interrupt("approve?")` pauses.
- `Command(resume="yes")` continues.
- Needs a checkpointer.
- `invoke(version="v2")` → `GraphOutput.interrupts`.

Interrupt rules — the gotchas.

- Node re-runs from the top on resume.
- No bare try / except around interrupt.
- Same order every run (index-matched).
- JSON-serializable payloads only.
- Side effects before interrupt → make them idempotent.

Static vs. dynamic interrupts.

`interrupt()`

dynamic — real HITL

`interrupt_before / after`

static breakpoints — debugging only

Time travel.

— `get_state_history()`

— Replay from any checkpoint.

— `update_state()` → fork a new branch.

Hands-on

Build a tool-calling agent — from zero.

Hello world.

```
g = StateGraph(MessagesState)
g.add_node("chat", chat_node)
g.add_edge(START, "chat")
g.add_edge("chat", END)
app = g.compile()
app.invoke({"messages": [...]}))
```

StateGraph(MessagesState) · one node · two edges · .invoke()

Bind a tool.

```
def search(q: str) -> str:                                     @tool
    ...

llm = model.bind_tools([search])
# agent node calls llm; tool node runs the calls
```

@tool · llm.bind_tools([...]) · a tool node + an agent node

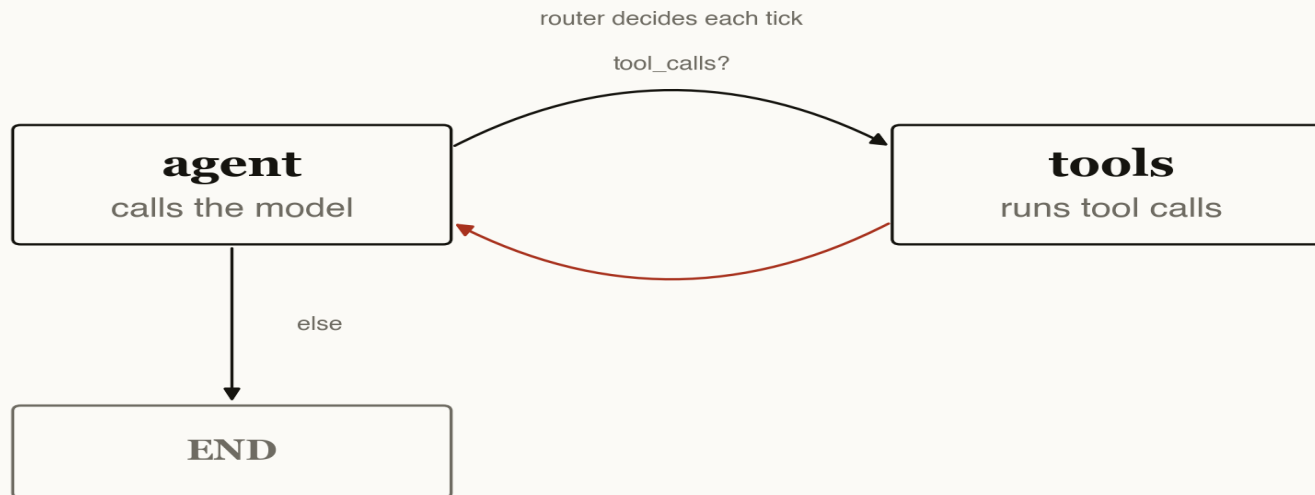
The loop.

- Router: tool_calls? → tools : END.
- The tools → agent edge closes the loop.
- This is your Class 6 while-loop, as a graph.

The picture.

The agent loop

Your Class 6 while-loop — drawn as a graph.



agent $\rightleftharpoons$ tools, agent $\rightarrow$ END.

Memory.

- Compile with InMemorySaver.
- Invoke with a thread_id.
- The second turn remembers the first.

Human-in-the-loop.

- `interrupt()` before the tool fires.
- Inspect, approve, edit.
- Resume with Command.

Time travel.

- List the checkpoints.
-
- Rewind to before a bad step.
-
- Fork an alternate path.

Subgraphs + multi-agent.

- A subgraph is a graph used as a node.
- Shared keys → add as a node; different schemas → invoke inside a node.
- Persistence: None (per-call) · True (per-thread) · False (stateless).
- A supervisor routes to workers.
- Hand-offs via `Command(goto, graph=PARENT)`.

Streaming.

— `stream(version="v2")` – unified `StreamPart {type, ns, data}`.

— `modes:` `values` · `updates` · `messages` · `custom` · `checkpoints` · `tasks` · `debug`.

— `stream_events(version="v3")` – typed projections.

— `.messages` · `.values` · `.interrupts` · `.output`

Send — map-reduce.

fan out

to N copies at runtime

`Send(node, slice)`

from a conditional edge

Durable execution + fault tolerance.

- A persistent checkpointer is fault tolerance.
- Crash → resume same thread_id with None input.
- durability = "exit" / "async" / "sync".
- Wrap side-effects in @task — idempotent on replay.

LangGraph Platform.

— Agent Server — checkpointing & store, automatic.

— Studio — visual debugging.

— Deploy long-running, stateful agents.

Wrap

Where LangGraph sits.

And when not to reach for it.

Graph API vs. Functional API.

Graph

explicit nodes & edges

Functional

@entrypoint + @task, plain control flow

both

same runtime, persistence, streaming, HITL

Middleware (the LangChain layer).

— `create_agent` from `langchain.agents`.

— Hooks: `before_model` / `after_model` / `wrap_model_call` / `wrap_tool_call`.

— Also: `before_agent` / `after_agent`.

— Guardrails, summarization, PII redaction, retries, dynamic model/tool.

Structured output.

- `create_agent(response_format=Schema)`.
- ToolStrategy (any model) vs ProviderStrategy (native).
- Result in `["structured_response"]`.
- v1: agent state must be a TypedDict.

LangChain vs. LangGraph vs. Deep Agents.

LangChain

the prebuilt agent — create_agent

LangGraph

low-level orchestration

Deep Agents

planning / subagent harness on top

When NOT to use LangGraph.

- One-shot prompt — just call the model.
-
- Linear chain — a function is fine.
-
- Reach for it when you need: loops, state, HITL, durability.

What you got for free.

- Durable state.
- Pause / resume.
- Inspect any step.
- Rewind + fork.
- All from "compile with a checkpointer."

The one idea.

Make control flow visible.

The graph is the program.